Architecture, Implementation, and Evaluation of LLM Wiki Systems as Persistent Memory for Autonomous Agents

1. Definition and Fundamental Philosophy of LLM Wiki

An LLM Wiki is an agent-native, persistent, human-readable, and machine-traversable knowledge compilation framework that operates as a dynamic memory layer between unstructured source data and reasoning language agents. Unlike traditional retrieval mechanisms that organize knowledge into static, uncoordinated chunk vectors, an LLM Wiki transforms raw unstructured text into an interlinked network of structured Markdown files. This system serves a dual purpose: it acts as an auditable knowledge repository for human inspection while simultaneously functioning as a structured environment through which Large Language Model (LLM) agents execute tool-guided compositional reasoning.

At its core, the LLM Wiki philosophy rejects the paradigm of passive context injection. Instead of treating memory as a flat database queried by vector similarity, the LLM Wiki treats memory as an evolving, structured codebase or encyclopedic system. The knowledge base is continuously compiled, cross-referenced, and updated by the language model itself upon the arrival of new information. By organizing knowledge into canonical entity pages, hierarchical directory indices, and explicit bidirectional wikilinks, the LLM Wiki shifts the operational focus from single-shot context retrieval to interactive, multi-hop reasoning traversals.

2. Historical Origin and Theoretical Evolution

The conceptual lineage of the LLM Wiki spans two distinct phases: an initial pragmatic pattern articulated by practitioners and a subsequent formalization within agentic research literature.

The initial conceptualization was introduced by Andrej Karpathy, who described the LLM Wiki as a personal knowledge base architecture built around interlinked Markdown files. In this original formulation, the system addresses the memory-less nature of standard LLM interactions. Traditional Retrieval-Augmented Generation (RAG) forces language models to rediscover domain insights from raw documents on every query. Karpathy proposed an alternative workflow where the LLM incrementally ingests source materials, synthesizes core concepts, writes Markdown pages, maintains index files, cross-references entities, and records decision states over time. Under this operational model, the human user manages data sourcing and directional querying, while the LLM manages the structural synthesis, filing, and bookkeeping.

This practical design pattern was formally conceptualized and operationalized as a research paradigm by Ming et al. in their work titled *Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki*. Ming et al. elevated the LLM Wiki from an ad-hoc personal knowledge management trick to a formalized, self-evolving retrieval framework designed explicitly for multi-agent reasoning systems. Their research demonstrated that structured compilation and explicit traversal affordances resolve the fundamental scaling limits of vector similarity lookups,

establishing LLM Wiki as a competitive paradigm for complex, multi-hop, and structured document QA tasks.

3. Core System Architecture

The theoretical framework of the LLM Wiki rests upon three architectural pillars: Compilability, Composability, and Evolvability.

Compilability dictates that raw, heterogeneous documents are not ingested as flat, independent chunks. Instead, an LLM compiler transforms source texts into a structured knowledge tree composed of standardized, interlinked units. This process synthesizes entities, extracts explicit relational claims, generates category-level listings, and establishes bidirectional links. Compilability ensures that information density is maximized per unit file, reducing text redundancy while embedding topological structure directly into the file system.

Composability requires that the structured knowledge base expose standard tool-calling interfaces—specifically search, read, and traversal operations—to the reasoning agent. Because knowledge is encapsulated within explicit entity pages connected by functional links, retrieval is no longer a single black-box execution step. Instead, retrieval becomes a composable sequence of sub-operations where the agent formulates a plan, inspects starting nodes, follows structural wikilinks, evaluates intermediate evidence, and dynamically adjusts its investigation path.

Evolvability ensures that the knowledge base self-corrects over time. Knowledge bases compiled by language models are prone to structural and semantic degradation over sequential ingestion cycles. Evolvability is achieved through persistent feedback loops that identify compilation flaws, track root causes, store constraint rules in a persistent Error Book, and trigger multi-layered repair pipelines. This enables the knowledge base to self-correct and refine its own topological integrity over time.

The file system hierarchy reflects these architectural principles through a predictable directory structure designed for efficient navigation:

- **Global Index (index.md)**: Positioned at the root level, this file contains high-level domain summaries, universal system tags, and explicit paths to category indices.
- **Category Directories and Indices (/category/_index.md)**: Located within subdirectories such as /people/_index.md or /organizations/_index.md, these files function as dynamic tables of contents, listing member entities alongside brief descriptions and primary aliases.
- **Entity Pages (/category/Entity_Name.md)**: Modular Markdown documents representing specific concepts, individuals, locations, or system objects, integrated into the directory structure via bidirectional wikilinks.

4. Data Model and Canonical Knowledge Schema

To guarantee that autonomous agents can reliably parse and navigate the LLM Wiki, every page adheres to a standardized schema using structured Markdown headers and YAML frontmatter metadata.

```
---
title: "Entity Name"
aliases: ["Alias 1", "Alias 2"]
tags: ["category/subcategory", "domain_type"]
```

last_updated: "2026-05-25"
sources: ["source_doc_id_001",
"source_doc_id_04[span_75](start_span)[span_75](end_span)[span_106](start_span)[span_106](end_span)2"]

Below the YAML frontmatter, the page body is structured into semantic sections designed to separate executive summaries from fine-grained evidence and relationship graphs:

- **Summary Section**: A concise overview describing the entity's core definition, significance, and overarching status.
- **Key Facts & Attributes**: Bulleted, atomic assertions cross-referenced against primary source documents.
- **Relationships & Wikilinks**: Explicitly annotated links using syntax such as [[Entity_B]] or [[Entity_C|Custom Link Text]]. These wikilinks act as hyperlinked edges in a knowledge graph, supplying functional traversal affordances to the reading agent.
- **Source References**: Grounding citations that point back to raw input files, preserving verifiable provenance.

The inclusion of bidirectional wikilinks is a structural prerequisite. When the LLM compiler links [[Entity_A]] to [[Entity_B]], a reverse link or structural reference is appended to Entity_B's inverse relationship section. This guarantees that graph traversal can occur forward or backward, preventing orphaned leaves and isolated subgraphs.

5. Retrieval Dynamics: Retrieval-as-Reasoning vs. Retrieval-as-Lookup

The transition from conventional RAG architectures to the LLM Wiki represents a paradigm shift from Retrieval-as-Lookup to Retrieval-as-Reasoning. In conventional setups, retrieval is treated as a black-box ranking step where vector similarity determines a static context block. In contrast, Retrieval-as-Reasoning frames retrieval as an active, agent-controlled search process.

Paradigm Property	Retrieval-as-Lookup (Conventional RAG)	Retrieval-as-Reasoning (LLM Wiki)
Execution Mechanics	Single-shot, static vector similarity search	Multi-step agentic traversal over interlinked pages
Context Assembly	Top-[span_39](start_span)[span_39](end_span)[span_49](start_span)[span_49](end_span)k raw text chunks concatenated into prompt	Progressive, agent-selected reading based on partial state
Intermediate Decisioning	None; context is locked before generation begins	Dynamic; agent reads, evaluates sufficiency, and jumps
Knowledge Topology	Unstructured, disjointed chunk vectors	Interlinked hierarchical Markdown web
Traceability	Implicit similarity scoring	Explicit tool execution logs and wikilink chains

Under Retrieval-as-Reasoning, the agent interacts with the LLM Wiki through standardized tool

interfaces:

- wiki_search(query: str) -> List[PageMetadata]: Executes a hybrid multi-stage lookup. It searches titles, aliases, tags, and page descriptions before falling back to full-text search across page bodies. It returns candidate paths, entity summaries, and relevancy markers.
- wiki_read(paths: List[str]) -> Dict[str, PageContent]: Performs batch retrieval of complete Markdown pages or directory index files. The returned content includes structured text and explicit wikilinks, exposing new traversal paths for subsequent steps.

Equipped with these tools, an agent handles multi-hop inquiries using explicit traversal patterns:

1. **Direct Access**: For simple factual queries, the agent issues wiki_search, directly identifies the target entity page, and reads the solution using wiki_read.
2. **Bridge Queries (A \rightarrow B \rightarrow \tex[span_87](start_span)[span_87](end_span)[span_118](start_span)[span_118](end_span)t{Answer})**: For questions requiring relational multi-hop connections (e.g., determining an attribute of an entity related to a primary subject), the agent searches for Entity A, reads Entity_A.md, identifies an explicit wikilink pointing to Entity_B.md, and calls wiki_read(["Entity_B.md"]) to extract the final evidence without requiring intermediate search operations.
3. **Exploratory Browsing**: For broad or structural queries, the agent reads high-level directory indices (_index.md), scans category groupings, selects promising entity nodes, and traverses down subtrees until evidence sufficiency is satisfied.

To prevent infinite loops during traversal, the execution framework enforces strict operational bounds: a maximum tool invocation budget T_{\max} = 15 and a patience threshold P = 3 for consecutive unproductive searches.

6. Knowledge Compilation and Incremental Ingestion

The transformation of unaligned raw text into an interlinked LLM Wiki is handled by an automated compilation pipeline. Rather than replacing or re-indexing the entire corpus upon receiving new documents, the LLM compiler executes an incremental ingestion sequence.

The ingestion workflow begins when a new unstructured document is ingested into the processing queue. The system executes document parsing and entity extraction, isolating key terms, relationships, and temporal markers. Next, the system calls a SELECTPAGES candidate identification routine. The compiler scans existing directory indices (_index.md) and global tags to select relevant pre-existing entity pages that must be updated, expanded, or cross-referenced.

Once targets are identified, the system invokes the COMPILEWIKIPAGES generation engine. The engine generates modified Markdown content for existing pages and drafts new entity pages for newly discovered concepts. During generation, the compiler embeds bidirectional wikilinks ([[...]]), updates categorical tables of contents, and resolves naming collisions or synonymous aliases. The compiled output undergoes structural and semantic validation to confirm that wikilinks target existing paths, frontmatter tags follow schema conventions, and citations match source identifiers. Finally, the revised Markdown tree is committed to the file system, and an operational entry is appended to the ingestion log.

7. Self-Evolving Mechanics and the Error Book Engine

A key limitation of automated knowledge base generation is structural and semantic degradation

over time. Compilers can introduce dangling links, unsupported claims, cross-page contradictions, and index drift. The LLM Wiki architecture addresses this through an explicit, self-evolving system anchored by a persistent Error Book.
Compilation failures fall into two structural classes:

- **Structural Validity Errors**: Systemic syntactic defects including dangling wikilinks (links pointing to non-existent entity files), malformed reference markers, incomplete frontmatter metadata, unseen overwrites (accidental deletion of existing facts during an update), and index inconsistencies. Structural syntax failures—specifically dangling links and malformed citations—account for the vast majority of compilation bugs.
- **Content-Level Consistency Errors**: Semantic flaws including unsupported factual statements (claims lacking source references) and cross-page contradictions (where two entity pages assert conflicting facts).

The self-correction mechanism processes compilation failures through a five-stage lifecycle: \text{Discover} \longrightarrow \text{Attribut[span_210](start_span)[span_210](end_span)[span_221](start_span)[span_221](end_span)e} \longrightarrow \text{Constrain} \longrightarrow \text{Inject} \longrightarrow \text{Verify \& Close}
The lifecycle begins with Discovery, where static analysis scripts and runtime validation agents identify structural or semantic defects within the file tree. During Attribution, the diagnostic module identifies the root cause of the compilation failure, such as a failure to update an inverse link or improper alias handling. In the Constraint stage, the system formalizes the root cause into explicit natural-language constraint rules (e.g., enforcing that creating a link to a new entity must instantiate the corresponding file in the directory index). These rules are logged in the persistent Error Book. During Injection, stored constraint rules from the Error Book are dynamically injected into future compilation system prompts, conditioning the compiler to avoid known failure modes. Finally, during Verification and Closure, the system runs a repair check, marking the error entry as resolved once compliance is verified.
To balance computational efficiency with structural integrity, repair operations are split into a two-layer pipeline:

- **Layer 1: Deterministic Code Auto-Fix**: Executes immediately after every ingestion batch. This layer uses deterministic Python routines to resolve syntax issues, such as pruning dangling links, creating placeholder files for uninstantiated entity links, re-formatting malformed metadata, and validating file paths.
- **Layer 2: LLM Periodic Fix**: Scheduled as a background job or triggered periodically. It uses full LLM inference to resolve complex semantic issues, including reconciling cross-page contradictions, synthesizing missing entity summaries, and resolving complex relational ambiguities.

8. Token Cost Dynamics and Context Window Efficiency

A major operational bottleneck in agentic RAG systems is token consumption and context window pollution. Concatenating large volumes of raw text chunks into prompt windows degrades long-context reasoning performance and increases token costs. The LLM Wiki optimizes context window efficiency through progressive information disclosure. Instead of loading large volumes of raw text, the agent selectively requests small, hyper-dense entity pages (200-500 tokens per page).

Let L_{raw} represent the average token length of retrieved raw document passages in conventional RAG (1,000-2,000 tokens per passage), and let k be the number of top retrieved candidates ($k \approx 10$-20). The total context footprint for traditional RAG scales as:
$$\text{Footprint}_{\text{RAG}} = \mathcal{O}(k \cdot L_{\text{raw}})$$
In contrast, the context footprint of an LLM Wiki traversal depends on the agent's exploration depth d (typically 2-4 hops) and the average length of a compiled entity page L_{wiki} (200-500 tokens):
$$\text{Footprint}_{\text{Wiki}} = \mathcal{O}(d \cdot L_{\text{wiki}})$$
Because $L_{\text{wiki}} \ll L_{\text{raw}}$ and $d \ll k$, the cumulative token footprint per query is significantly smaller in an LLM Wiki, even across multi-turn tool calling loops.
Although the LLM Wiki incurs upfront token costs during initial compilation, this expenditure is amortized over subsequent queries. Furthermore, because knowledge is stored as static, human-readable Markdown files, pages can be cached at the file system level or within key-value prompt caches, significantly reducing inference latency.

9. Structural Failure Modes and Quality Maintenance

Despite its structured architecture, the LLM Wiki is susceptible to dynamic failure modes that require active maintenance:

- **Link Rot and Dangling Nodes**: Occurs when an LLM compiler references an entity within a generated text block without instantiating the corresponding file or updating the directory index. This creates broken traversal paths that stall agentic execution.
- **Index Drift and Categorical Stagnation**: As new entity pages are added, directory indices (_index.md) can become outdated if not updated concurrently. An agent searching the directory index may miss newly added entities.
- **Unseen Overwrites and Fact Erasure**: During incremental updates, an LLM compiler may overwrite an existing page to add new information, inadvertently deleting historical facts or pre-existing wikilinks.
- **Semantic Contradictions Across Pages**: If input sources contain contradictory claims, an LLM compiler may record conflicting assertions in separate entity pages.

To mitigate these quality failure modes, operational production deployments must implement strict automated safeguards. Post-compilation deterministic path hooks intercept file system commits and automatically resolve missing entity pages by generating stub pages or stripping invalid links. Storing the LLM Wiki in a Git repository enables structural diffing, allowing the system to reject updates that delete large blocks of pre-existing text or structural links. Additionally, automated background workers scan interlinked pages for explicit numerical or temporal assertions, flagging contradictions for resolution by the Layer 2 LLM periodic repair pipeline.

10. Comparative Analysis: LLM Wiki vs Contemporary Memory Architectures

To contextualize the performance and trade-offs of the LLM Wiki architecture, the following table compares it against existing retrieval paradigms across core operational dimensions:

Feature / Axis	Dense Vector RAG	GraphRAG / LightRAG	HippoRAG 2	Key-Value / Buffer Memory	**LLM Wiki**
Knowledge Representation	Unstructured flat text vectors	Subject-Predicate-Object triples / Graphs	Personalized PageRank Knowledge Graph	Raw conversation strings / Key-Value state	**Interlinked Markdown file hierarchy (.md)**
Context Window Footprint	Large (dumps multiple raw chunks)	Medium-Large (graph subtrees + text)	Medium (graph neighborhood extractions)	Linear expansion with history	**Low / Progressive (loads minimal pages)**
Reasoning Traversal Method	Top-k similarity matching	Graph walk / Global community summary	Graph-based personalized PageRank	Direct key lookups or string match	**Agent-guided explicit link traversal**
Incremental Update Cost	Low (embed & insert into vector index)	High (re-extract triples & recalculate graphs)	High (re-index graph relationships)	Low (append string/buffer)	**Medium (selective compilation & link update)**
Self-Correction Mechanism	None (static lookup)	Limited (graph pruning routines)	Implicit graph weight adjustment	None	**Explicit 5-Stage Error Book Loop**
Human Readability & Auditability	Poor (opaque high-dimensional vectors)	Moderate (requires specialized graph viewers)	Moderate (graph visualization needed)	High (raw text)	**High (standard Markdown file system)**
Multi-Hop Task Generalization	Weak (suffers from semantic drift)	Strong on graph chains; weak on unstructured context	Strong on specific relational subgraphs	Poor for deep cross-domain lookups	**State-of-the-art across structured & multi-hop QA**

Analytically, while dense vector systems offer fast ingestion, they suffer from semantic drift during multi-hop reasoning because vector closeness does not imply logical connectivity. Graph-based systems resolve connectivity issues but introduce high computational overhead during graph generation, rendering incrementally added data expensive to process. The LLM Wiki balances structural representation with practical maintainability: its interlinked Markdown structure provides explicit relational edges without requiring custom graph database infrastructure, while its standard file format ensures complete auditability by human operators.

11. Critical Evaluation of Key Research Papers

The theoretical foundation of the LLM Wiki paradigm is primarily established by Ming et al. in *Retrieval as Reasoning: Self-Evolving Agent-Native Retrieval via LLM-Wiki*. Ming et al. established that standard RAG implementations suffer from an architectural mismatch when paired with autonomous tool-using agents. They demonstrated that treating retrieval as a single-shot vector lookup deprives agents of the opportunity to inspect intermediate evidence, refine search criteria, and actively explore knowledge topologies.

To validate the framework, Ming et al. conducted evaluations against major memory

architectures—including Dense RAG, GraphRAG, LightRAG, and HippoRAG 2—across multi-hop benchmarking datasets:

- **HotpotQA, MuSiQue, and 2WikiMultiHopQA**: The LLM Wiki architecture achieved consistent gains, outperforming the strongest graph-based baselines (HippoRAG 2, GraphRAG, LightRAG) by 2.0 to 8.1 F1 points. The improvements scaled proportionally with hop count, confirming that explicit link traversal mitigates the error propagation typical of multi-hop vector searches.
- **AuthTrace Evaluation**: On AuthTrace—a benchmark measuring complex, multi-document structured queries—LLM Wiki recorded the highest overall accuracy. This demonstrated that compilation-based knowledge organization generalizes beyond chain-style relational reasoning to complex, highly structured domain spaces.

Ablation studies conducted by Ming et al. isolated the performance contributions of each core module:

1. Removing the Wiki file structure in favor of flat text chunks caused a performance degradation, proving the utility of canonical page aggregation.
2. Replacing progressive tool traversal with single-shot top-k selection dropped multi-hop accuracy, confirming the core hypothesis of the Retrieval-as-Reasoning paradigm.
3. Disabling the Error Book mechanism resulted in a continuous decay in structural link integrity and a 3.4 point drop in F1 across multi-batch ingestion benchmarks.

Parallel research on self-reasoning frameworks (e.g., AAAI literature on self-reasoning RALM) supports these conclusions. These studies emphasize that forcing language models to explicitly generate, verify, and traverse intermediate citation trajectories improves answer robustness and mitigates hallucination compared to implicit, ungrounded vector context injection.

12. Production Implementation Recommendations

For engineering teams deploying an LLM Wiki in enterprise environments, system design decisions should align with the following best practices:

- **Storage Backend**: Store the LLM Wiki as a native file directory of UTF-8 Markdown files backed by Git. Git provides deterministic versioning, atomic commits, branch isolation for parallel compilation tasks, and audit logs.
- **Hybrid Search Engine**: Implement wiki_search as a hybrid search engine combining vector embeddings (for semantic fallback) with keyword BM25 indexing over YAML frontmatter metadata (titles, aliases, and tags). Alias matching must take precedence to ensure deterministic routing to target entity pages.
- **Concurrency Control & Synchronization**: When multiple background agents update the Wiki concurrently, apply file-level mutex locks or queue incoming document ingestion through a serial compilation worker pipeline to prevent race conditions during updates.
- **Tool Interface Sandboxing**: Expose the wiki_search and wiki_read operations to agents via standardized, strictly typed JSON function signatures. Restrict agent tool access to prevent direct write operations during retrieval phases.
- **Pre-Commit Quality Hooks**: Deploy a Layer 1 code auto-fix pipeline as a pre-commit hook that validates link targets, verifies frontmatter schema definitions, and enforces bidirectional consistency across the directory tree.

13. Case Study: RPG and World Simulation Applications

A compelling application for the LLM Wiki architecture is complex role-playing games (RPGs) and multi-agent world simulations. Traditional world simulation systems struggle with long-term

memory maintenance; as simulated worlds expand, storing character histories, location state changes, active quests, and relational dynamics in simple context windows or standard vector databases leads to state drift, temporal contradictions, and forgotten lore.
An LLM Wiki resolves these issues by serving as an active state graph and living encyclopedia for the simulated world. The directory hierarchy categorizes simulation elements into distinct subdirectories:

- **Root Directory (/world_root/index.md)**: Contains the global world overview, physics or magic system constraints, current campaign epoch, and overarching history.
- **Location Directory (/world_root/locati[span_97](start_span)[span_97](end_span)[span_128](start_span)[span_128](end_span)ons/)**: Houses regional indices (_index.md) and specific entity pages (e.g., Ironforge_Citadel.md), tracking control status, current political leadership, local non-player characters (NPCs), and environmental state variables.
- **Character Directory (/world_root/characters/)**: Stores character registries (_index.md) and individual character pages (e.g., Eldrin_The_Mage.md), recording current inventory, emotional disposition toward players, faction loyalty metrics, and historical interaction logs.
- **State Logs Directory (/world_root/state_logs/)**: Tracks dynamic world states, such as active_quests.md for objective trees and dec[span_142](start_span)[span_142](end_span)ision_history.md for a chronological ledger of player decisions and historical milestones.

When a player or agent interacts with the simulation, the system executes structured state reads and writes. For state updates (ingestion), if a major event occurs (e.g., a player alters a region's political power), the LLM compiler processes the event log, updates the location page, revises the affected character pages, and appends an entry to the decision history log. For state querying (retrieval), when an NPC generates dialogue or plans an action, the NPC agent calls wiki_read on its own character page, follows interlinked references to its current location, and inspects relevant active quest files. This allows the NPC to generate contextually accurate behaviors grounded in verified world state without polluting its active context window with unrelated global lore.

14. Token-Efficient RPG Retrieval Strategies

To operate world simulations under strict real-time latency constraints, system architects must enforce strict token-efficient traversal strategies for world simulation agents:

1. **Category Index Pruning**: Agents must query regional or categorical indices (locations/_index.md) prior to loading deep entity pages, determining target paths with minimal initial token load.
2. **Context Budget Allocation**: Limit the agent's working context during turn generation to a strict allocation:
   \text{Context Budget} = \text{System Prompt} (1,000\text{t}) + \text{Agent Page} (300\text{t}) + \text{Current Location [span_149](start_span)[span_149](end_span)Page} (400\text{t}) + \text{Active Traversal Pages} (\le 800\text{t})
1. **Subtree Scoping**: Force dialogue agents to execute local graph traversals bounded to a maximum distance of H \le 2 hops from their current location and social network nodes.
2. **Cached Frontmatter Headers**: Cache frontmatter metadata summaries for fast evaluation, reading full Markdown page bodies only when explicit sub-state validation is

required.

15. Experimental Design for Benchmarking LLM Wiki

To evaluate an LLM Wiki deployment against alternative memory architectures within a custom domain, researchers should implement the following experimental protocol:

Benchmarking Datasets

Utilize a mix of standard multi-hop QA benchmarks alongside domain-specific structured evaluations:

- **Multi-Hop Relational QA**: HotpotQA, MuSiQue, and 2WikiMultiHopQA (evaluating multi-step chain-of-thought reasoning across disparate documents).
- **Structured Multi-Document Reasoning**: AuthTrace (measuring authorization tracing and multi-document organizational hierarchy lookups).

Performance Metrics

Quantitative evaluation relies on three metrics:

- **Exact Match (EM) & F1 Score**: Standard accuracy metrics evaluating response precision and recall against ground truth answers.
- **Traversal Efficiency Ratio (\text{TER})**: Computed as the total tokens consumed during retrieval divided by the final answer F1 score:

\text{TER} = \frac{\text{Total Tokens Consumed}}{\text{F1 Score}}

Lower \text{TER} values indicate superior context window efficiency per unit of accuracy.

- **Link Integrity Rate (\text{LIR})**: The percentage of wikilinks in the compiled dataset that point to valid, non-dangling entity files.

Experimental Control Matrix

Compare the target LLM Wiki framework against five control architectures using standardized LLM backends:

1. **Standard Dense RAG**: Chunk size 512 tokens, overlap 64, top-k \in \{5, 10, 20\}.
2. **GraphRAG**: Default community summary and graph traversal pipeline.
3. **HippoRAG 2**: Personalized PageRank graph-based retrieval.
4. **LLM Wiki (No Self-Correction)**: Compiled Markdown wiki operating without the Error Book loop.
5. **LLM Wiki (Full Framework)**: Complete operational system featuring progressive traversal and the 5-Stage Error Book pipeline.

Hypothesis testing should confirm that Full LLM Wiki achieves statistically significant gains in F1 score on multi-hop benchmarks while maintaining a lower \text{TER} compared to Dense RAG and GraphRAG baselines.

16. Final Verdict and Future Directions

The LLM Wiki architecture redefines long-term memory and knowledge retrieval for autonomous AI applications. By replacing single-shot, uncoordinated vector lookups with an agent-native,

compilable, interlinked Markdown file structure, the framework operationalizes the Retrieval-as-Reasoning paradigm.

Empirical evidence confirms that explicit wikilink traversal, structured directory indices, and persistent self-correction via the Error Book yield state-of-the-art results across complex multi-hop QA benchmarks. Furthermore, the system preserves human auditability, offers predictable file system management, and maximizes context window token efficiency.

As language model agents transition from passive chatbots to complex autonomous actors, the LLM Wiki provides a robust, self-evolving foundational memory layer. Future research directions will focus on scaling compilation mechanics to web-scale contexts and adapting the Error Book self-correction paradigm to real-time streaming data environments.

ผลงานที่อ้างอิง

1. Retrieval as Reasoning: Self-Evolving Agent-Native ... - alphaXiv, https://www.alphaxiv.org/abs/2605.25480 2. LLM Wiki - GitHub Gist, https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f 3. Self-Evolving Agent-Native Retrieval via LLM-Wiki - Moonlight, https://www.themoonlight.io/zh/review/retrieval-as-reasoning-self-evolving-agent-native-retrieval-via-llm-wiki 4. Self-Evolving Agent-Native Retrieval via LLM-Wiki - arXiv, https://arxiv.org/html/2605.25480v2 5. Karpathy's LLM Wiki - Build Your Knowledge Base - Get Claude Skills, https://www.getclaudeskills.com/skills/karpathy-s-llm-wiki-nousresearch 6. Self-Evolving Agent-Native Retrieval via LLM-Wiki | Cool Papers, https://papers.cool/arxiv/2605.25480 7. Improving Retrieval Augmented Language Model with Self-Reasoning, https://ojs.aaai.org/index.php/AAAI/article/view/34743/36898